



QuestKids

Project Overview & Handoff Document

Flutter

Firebase

Cloud Functions

Gemini AI

Written for handoff to another AI assistant

Last updated 2026-10-10

QuestKids — Project Overview

Written for handoff to another AI assistant (e.g. Gemini) who will make future updates on request. Last updated 2026-10-10.

1. What QuestKids Is

QuestKids is a gamified learning platform for South African primary school learners (Grades 1–7), built on the CAPS curriculum. It targets Android (primary), Web, and iOS, built in Flutter with a Firebase backend.

Three roles:

- **Learner** — plays curriculum games across 64 distinct game engines, earns XP/coins/badges, chats with "QuestBot" (an AI tutor, on-screen as "Quest Boy")
- **Parent** — links to one or more children (via QR/link-code or request approval), views progress analytics, verifies completed work, receives AI weekly reports and activity notifications
- **Teacher** — class-level analytics, assigns missions, verifies progress (scoped to their own class only — no global read access)

It is built and maintained by a solo developer on a memory-constrained Windows development machine (~8GB RAM) — this shapes some tooling choices (e.g. pinned Flutter/Java versions, capped Gradle JVM heap).

2. Tech Stack

Layer	Technology
App	Flutter (Dart SDK ≥ 3.4), Material 3
State management	Provider (<code>ChangeNotifierProvider</code> , registered in <code>lib/main.dart</code>)
Navigation	Classic <code>Navigator</code> named routes (not <code>go_router</code>)
Backend	Firebase: Auth, Firestore, Storage, Cloud Messaging, Analytics, Cloud Functions (v2, TypeScript)
AI	Google Gemini, proxied exclusively through Cloud Functions — no AI API keys ever reach the client
Text-to-speech	Google Cloud Text-to-Speech (QuestBot's voice), proxied the same way
Offline support	sqlite (+ <code>sqlite_common_ffl</code> for desktop), via <code>OfflineService</code>
Charts	<code>fl_chart</code>

4. Game Engine Architecture

```
GameRouter → <Engine>Game (widget) → <Engine>Session (state, optional) → GameEngine (pure
```

There are **two real architectural patterns** in the codebase today, not one:

1. **The original 8 "fully layered" engines** (Tug of War, Adventure Journey, Runner Collector, Explorer Map, Multiples Merge, Sequence Builder, Circuit Builder, Budget Builder) — these have a separate `GameEngine` (pure Dart, no Flutter imports), a `GameSessionState` subclass, and a thin `Game` widget. All mutable state lives in the Session class.
2. **56 self-contained engines** — a single `StatefulWidget` per engine with its own internal phase-enum state machine (`intro/question/correct/wrong/ streak/...`), no separate Engine/Session split. This is the dominant pattern for everything added after the original 8, including `number_counting_duel` (CLAUDE.md describes it as one of the "original 9 layered" engines, but in the actual code it is self-contained — a known, harmless doc/code drift, not yet corrected).

64 engines total, registered in `GameRouter` 's switch statement and made visible to players only via entries in `game_catalog.dart` (126 catalog entries map grade/subject/topic combinations onto these 64 engines — an engine with no catalog entry is built but invisible).

All 64 now call `GameFeedbackService.correct()` / `.incorrect()` for spoken answer feedback (as of 2026-10-10 — previously only the original 8 did; see §8).

5. Security Model

Role-based access control via Firebase Auth custom claims, never a client-writable field:

- `request.auth.token.role` is the *only* thing Firestore/Storage rules ever trust for authorization decisions.
- The `role` field mirrored onto a user's own Firestore document is for display/query convenience only.
- Roles are granted exclusively by Cloud Functions using the Admin SDK (`setUserRole` , `bootstrapAdmin` , `grantSelfDeclaredRoleClaim` , `assignDefaultRole`) — no client write to a role claim is ever possible.
- Exactly one `admin` claim exists in production today (audited 2026-10-10, confirmed correct).

Firestore rules (`firestore.rules`) are collection-by-collection allow-lists; anything not explicitly matched is denied by default (confirmed — there is no catch-all permissive rule). Notable hardened paths, each with an inline comment explaining the exact exploit it closes:

- `users/{uid}` create requires either a full POPIA consent trail (learner) or a null `birthDate` (parent), and `linkedChildrenUids.size() == 0` on every branch — closing a role-escalation path

where a hostile client could pre-populate access to another user's data before any admin ever grants them a `parent` claim.

- `progress/{id}` update splits "owner can update their own non-verified fields" from "a *linked* parent can verify, but only `verified / proofUrl`" — a prior version let any parent verify any child's progress.
- `emails/{id}` is fully `allow read, create, update, delete: if false` — previously any signed-in client (including a child) could write arbitrary `to / template / data`, an open-relay + HTML-injection vector via the `sendEmail` trigger. HTML-escaping was independently verified in `sendEmail` itself as defense-in-depth.
- `game_sessions`, `usage_ai`, `usage_tts`, `otp_challenges`, `security_login_attempts` are all `allow write: if false` — every one of these is server-authoritative, written only via the Admin SDK.

Rate limiting / abuse control:

- Child login (`loginChild`): per-name+IP and per-IP hashed buckets (SHA-256, never stores a raw name or IP), 10 and 60 attempts per 15-minute window. Verified under true concurrent load — a 2026-10-10 fix ensures a Firestore transaction-contention error during a burst of simultaneous requests is never surfaced to the caller as a raw internal error; it now always comes back as the same clean "too many attempts" message.
- AI usage (`usage_ai/{uid}`): 50 Gemini calls/day shared across all 6 `gemini/proxy.ts` functions (not per-function).
- AI parent insights (`usage_parent_ai/{uid}`): 10/day, enforced even on the path that never calls Gemini (zero-scored-sessions deterministic response) — cost control holds regardless of whether the AI is actually invoked.
- Text-to-speech (`usage_tts/{uid}`): 200/day, a separate counter from the chat quota.

Step-up email OTP (new, 2026-10-10): sensitive, single-button parent actions can require a one-time emailed code before proceeding. Currently wired into `unlinkParentChild` only (see §8 for why `setParentPermissions` was deliberately left out). The code is SHA-256-hashed at rest, 6 digits, expires in 5 minutes, single-use, locked out after 5 incorrect guesses, with a 60-second resend cooldown.

6. The Five Phases (what was built, what's actually verified)

Phase	What it is	Status as of 2026-10-10
1 — Account security	Child login, role management, admin bootstrap, OTP step-up	Deployed; auth/role Functions well-tested; OTP implemented for one action; full admin-account audit clean (exactly one admin)
2 — Game-answer feedback	Spoken "Correct!/Try again!" via <code>GameFeedbackService</code>	All 64 engines wired (was 8/64 until this session's audit)

Phase	What it is	Status as of 2026-10-10
3 — Parent notifications	Badge awards, quest reminders, game-activity updates, inactivity nudges, push delivery	Deployed; all 5 notification Functions behaviorally tested, including the inactivity idempotency guarantee (a repeated daily run never re-sends an already-fired threshold)
4 — AI parent insights	<code>getParentLearningInsights</code> — Gemini-generated, server-computed-only suggestions from real game session data	Deployed; now tested (cross-family isolation, zero-session deterministic fallback, quota enforcement)
5 — Admin platform reports	<code>getAdminPlatformReport</code> — aggregate counts for the admin dashboard	Deployed; tested since before this session

7. Testing Infrastructure

- **Cloud Functions:** Jest against the real local Firebase emulator (Firestore + Storage + Auth, never production). **35 of 36 exported functions have behavioral tests** (286 tests total). The only one that doesn't: `assignDefaultRole` — the SDK's `BlockingFunction` type has no `.run()` test hook; a real test would mean simulating Identity Platform's signed-JWT blocking-function HTTP contract for a one-line static return value, which isn't proportionate. External services with no emulator (`@google/generative-ai` , `@google-cloud/text-to-speech` , `nodemailer`) are mocked so the surrounding business logic — quota, auth, validation, error handling — is verified without a real paid API call or a real email send.
- **Firestore/Storage rules:** a dedicated negative-test suite (`@firebase/rules-unit-testing` , 50+ tests) asserting every previously-documented security fix stays denied.
- **Flutter:** 406 widget/unit tests, `flutter analyze` at 0 issues.
- **CI:** GitHub Actions runs both the Flutter and Functions suites on every push to any branch.

8. Known Gaps and Suggestions for Future Work

This section is the most useful one for an AI assistant picking this project back up — it names exactly what's unfinished and why, so work isn't duplicated or built on a wrong assumption.

Not yet done

1. `setParentPermissions` **OTP step-up** — deliberately not gated the same way as `unlinkParentChild` . Its screen (`parent_access_screen.dart`) saves on every individual permission-switch toggle; gating that with a step-up code would mean an OTP prompt per toggle flip. This needs a "batch changes, confirm once" redesign of that screen before OTP can be added sensibly.

2. **Rate-limit concurrency testing has a known methodology ceiling:** firing many simultaneous requests from one Node process sharing one Firestore client (the only way to test this locally) produces genuinely non-deterministic results from the *emulator's* transaction-retry behavior under that specific adversarial pattern — this isn't representative of how independent Cloud Functions instances behave against production Firestore. What's actually verified is the fix that matters: no raw internal error ever reaches the caller. True production-scale concurrent-load testing would need a real staging environment, not the local emulator.
3. **AI prompt-injection resistance** is verified only at the code layer (history role-filtering, truncation, no client-controlled field reaching the system prompt) — whether the model itself resists a cleverly-worded adversarial message is a property of Gemini's own behavior, which cannot be verified deterministically without a real, non-deterministic, paid API call. If this matters more than the current code-layer defenses provide, consider periodic manual red-teaming of the actual deployed `questyChat` endpoint.
4. **Staging/manual walkthrough** — no environment has been walked through interactively end-to-end (admin login, parent-child linking, a full game session, push notification delivery) in a real browser/device. This requires either a disposable staging Firebase project or a carefully scoped production dry-run with test accounts, neither of which has been set up.
5. **Two parallel "approve a parent link request" code paths exist:** `approveParentLinkRequest` (older, simpler) and `resolveParentLinkRequest` (newer, handles approve/decline/cancel with role-revalidation and default permissions). Both are exported and both work as tested, but having two endpoints doing overlapping jobs is worth consolidating — confirm with the Flutter client which one the UI actually calls today, then consider deprecating the other.
6. **A duplicate email-verification pattern:** `otp_code` (new, for action step-up) sits alongside the pre-existing `email_verification` template — these serve different purposes today but are worth keeping distinct intentionally rather than by accident as the OTP feature potentially grows to cover more actions (signup verification, password reset) later.

Suggestions for what to add next

- **Extend the OTP step-up to account recovery.** The infrastructure (`requestActionOtp` / `verifyAndConsumeActionOtp` in `functions/src/parent/actionOtp.ts`) is already generic — adding a new entry to `SENSITIVE_PARENT_ACTIONS` and a matching Flutter dialog call is a small lift for a real security win (password reset without relying solely on a clickable link).
- **A teacher-facing equivalent audit.** This session's work focused on parent/learner/admin Functions. Teacher-scoped reads/writes (`getFirestore()` calls scoped by `classId` / `teacherId`) haven't had the same behavioral-test pass.
- **Rules-level tests for the subcollections under `game_level_progress` and `game_progress`** — covered at a high level in the current rules suite but not exhaustively per-engine.
- **A lightweight Firebase-emulator wiring for Flutter widget/integration tests.** Right now, every screen that calls a Cloud Function (`parent_access_screen.dart` , `child_analytics_screen.dart` , etc.) has zero Flutter-side test coverage of that call — the business logic is proven on the Functions side, but the UI glue code calling it isn't. Adding `useFunctionsEmulator` / `useFirestoreEmulator` / `useAuthEmulator` calls behind a debug flag in

`main.dart`, plus a documented way to point a test run at the local emulator suite, would close this gap for every future feature, not just OTP.

- **Resolve the** `number_counting_duel` **doc/code drift** noted in §4 — either correct `CLAUDE.md` to describe it accurately, or actually refactor it onto the layered `GameEngine` / `GameSessionState` pattern to match what the doc claims.
-

9. Where to Find More Detail

- `docs/QUESTKIDS_COMPLETION_AND_RELEASE_REPORT.md` — the full session-by-session audit trail this overview is distilled from: every fix, every test added, every bug found and why, with commit hashes and CI run links. Read this for the "how do I know this is true" behind every claim above.
- `CLAUDE.md` (repo root) — the architecture/convention reference for anyone (human or AI) working in this codebase day to day.
- `firestore.rules` / `storage.rules` — read before touching any collection; the inline comments document *why* each rule exists, often referencing a specific exploit it closes.